

넘파이(NumPy) 기초 강의 교재

파이썬 판다스 기초 강의 | 조대표

1. 넘파이(NumPy)란?

[개념] 넘파이는 파이썬으로 과학 계산을 하는 기본적인 패키지입니다. 주로 행렬(행과 열로 구성된 2차원 배열) 또는 다차원 배열을 파이썬에서 효과적으로 다룰 때 사용합니다.

2. 넘파이 импорт 명령어

Python

```
import numpy as np # 가장 많이 사용하는 방식
```

방법	코드	설명
기본 импорт	<code>import numpy</code>	numpy. 접두사로 사용
관례적 импорт	<code>import numpy as np</code>	np. 로 축약 — 코드 간결, 가독성 향상

[TIP] 거의 모든 개발자가 `import numpy as np` 방식을 사용합니다.

3. 파이썬 리스트 vs 넘파이 배열의 차이

구분	파이썬 리스트	넘파이 ndarray
자료구조 타입	일반 리스트	다차원 배열 클래스
요소 연산	반복문(for문) 필요	벡터화 연산 가능 (반복문 없이)
연산 속도	느림 (파이썬 인터프리터)	빠름 (C언어로 구현)
인덱싱 방식	행 단위만 기본 지원	행, 열 모두 인덱싱 가능

4. 벡터화(Vectorization)란?

[개념] 벡터화란 반복문(for문) 없이 배열의 모든 원소에 동시에 연산을 적용하는 방식입니다. 마치 엑셀에서 수식 하나를 드래그하면 전체 셀에 자동 적용되는 것과 같습니다.

비교

```
# 일반 방식 (for문) | 벡터화 방식 (넘파이)

data = [1, 2, 3, 4] | arr = np.array([1, 2, 3, 4])

for i in data: | result = arr * 10

result.append(i * 10) | # [10 20 30 40] - - 한 번에 처리!
```

항목	설명
코드 간결성	for문 없이 수식 한 줄로 표현
속도	C언어 기반 처리 — for문보다 수십~수백 배 빠름
가독성	수학 수식과 유사한 형태로 직관적

5. 배열 생성과 벡터화 연산 예시

[X] 비효율적 방법 — 파이썬 리스트

Python List

```
data = [1, 2, 3, 4]

result = []

for i in data:

    result.append(i * 10)

print(result) # [10, 20, 30, 40]
```

- 리스트에 직접 * 10 을 하면 값이 반복될 뿐, 각 원소에 곱해지지 않음
- 원소별 연산을 위해 반복문이 필요 — 코드가 길고 불편함

[O] 넘파이 벡터화 연산 (1차원)

NumPy 1D

```
import numpy as np

arr = np.array([1, 2, 3, 4]) # 리스트 -> ndarray 변환

print(arr * 10) # [10 20 30 40]

print(arr + 5) # [ 6  7  8  9]
```

```
print(arr ** 2) # [ 1 4 9 16]
```

```
print(arr * arr) # [ 1 4 9 16] < - 원소끼리 곱셈
```

[O] 2차원 배열 벡터화 연산

NumPy 2D

```
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr2 * 2) # [[ 2 4 6] [ 8 10 12]] < - 모든 원소에 적용
```

```
print(arr2 + 10) # [[11 12 13] [14 15 16]]
```

```
print(arr2 ** 2) # [[ 1 4 9] [16 25 36]]
```

[핵심] 1차원이든 2차원이든 넘파이 배열이면 벡터화 연산이 동일하게 적용됩니다.

6. 넘파이 배열 생성 및 초기화 함수

배열 생성 함수 모음

```
np.array([1,2,3]) # 리스트 -> ndarray

np.zeros((2, 3)) # 0으로 채운 2x3 배열

np.ones((2, 3)) # 1로 채운 2x3 배열

np.full((2, 3), 7) # 7로 채운 2x3 배열

np.eye(3) # 3x3 단위 행렬

np.arange(0, 10, 2) # [0 2 4 6 8] (시작, 끝, 간격)

np.arange(0, 1, 0.1) # [0. 0.1 0.2 ... 0.9] (소수점 간격)

np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.] (균등 n개)

np.random.rand(2, 3) # 0~1 랜덤 실수 배열

np.random.randint(1,10,(2,3)) # 랜덤 정수 배열
```

함수	설명	예시
np.array()	리스트 -> ndarray 변환	np.array([1,2,3])
np.zeros()	0으로 채운 배열	np.zeros((2,3))
np.ones()	1로 채운 배열	np.ones((4,5))
np.full()	지정값으로 채운 배열	np.full((2,3), 7)
np.eye()	단위 행렬	np.eye(3)
np.arange()	범위+간격으로 생성	np.arange(0,10,2)
np.linspace()	균등 간격 n개 생성	np.linspace(0,10,20)
np.random.rand()	0~1 랜덤 실수 배열	np.random.rand(2,3)
np.random.randint()	범위 내 랜덤 정수 배열	np.random.randint(1,10,(2,3))

7. reshape — 배열 형태 변경

[개념] reshape()은 배열의 원소 수는 그대로 유지하면서 행과 열의 구조를 바꾸는 함수입니다. 원소 수가 맞지 않으면 ValueError가 발생합니다.

reshape 예제

```
import numpy as np

# 1차원 -> 2차원 변환

data = [0, 1, 2, 3, 4, 5]

arr = np.array(data).reshape(2, 3)

print(arr)

# [[0 1 2]
#  [3 4 5]]

# 2차원 -> 1차원 변환

arr2d = np.array([[0,1,2],[3,4,5]])

print(arr2d.reshape(6)) # [0 1 2 3 4 5]

# 1차원 -> 열벡터 (3행 1열)

arr = np.arange(3)

print(arr.reshape(3, 1))

# [[0]
#  [1]
#  [2]]

# reshape 오류 예시 (원소 수 불일치)

# np.arange(6).reshape(2, 4) -> ValueError!

# 원소 6개를 2x4(=8개)로 바꿀 수 없음
```

[주의] reshape 규칙: 변환 전후의 원소 총 수가 반드시 같아야 합니다. 예) 6개 원소 -> (2,3) OK / (2,4) ERROR (8개 필요)

8. dtype과 astype — 데이터 타입

[개념] dtype은 배열에 저장된 데이터의 타입을 나타냅니다. astype()으로 타입을 변환할 수 있습니다.

dtype / astype 예제

```
import numpy as np

# dtype 확인

arr = np.array([1, 2, 3])

print(arr.dtype) # int64 (정수형)

arr_str = np.array(['1', '2', '3'])

print(arr_str.dtype) # <U1 (문자열)

arr_float = np.array([1.1, 2.1, 3.1])

print(arr_float.dtype) # float64 (실수형)

# astype으로 타입 변환

arr = np.array([1, 2, 3])

arr_int16 = np.array(arr, dtype=np.int16)

print(arr_int16.dtype) # int16

# float64 -> int32 변환 (소수점 버림)

arr = np.array([1.1, 2.1, 3.1])

print(arr.astype(np.int32)) # [1 2 3]
```

dtype	설명	예시
int32 / int64	정수형 (32bit / 64bit)	np.array([1,2,3])
float32 / float64	실수형	np.array([1.1, 2.2])

dtype	설명	예시
<U1, <U10 등	문자열 (유니코드)	np.array(['a','b'])
bool	불리언 (True/False)	np.array([True, False])

9. 2차원 배열 인덱싱

행/열 인덱싱

```
import numpy as np

arr = np.arange(9).reshape(3, 3)

# [[0 1 2]
#  [3 4 5]
#  [6 7 8]]

print(arr[0]) # 첫 번째 행: [0 1 2]

print(arr[2]) # 마지막 행: [6 7 8]

print(arr[:, 1]) # 두 번째 열: [1 4 7]

print(arr[0, 0]) # 1행 1열: 0

print(arr[3, 2]) # 4행 3열: 17 (5x5 배열 기준)
```

인덱싱 문법	의미
arr[행]	해당 행 전체 반환
arr[행, 열]	특정 위치 원소 반환
arr[:, 열]	모든 행에서 특정 열 반환 (파이썬 리스트 불가)

10. 슬라이싱(Slicing)

[개념] 슬라이싱은 배열의 일부 구간을 선택하는 방법입니다. `arr[시작:끝]` 형태로 사용하며, 끝 인덱스는 포함되지 않습니다.

슬라이싱 예제

```
import numpy as np

# 1차원 슬라이싱

arr = np.arange(5) # [0 1 2 3 4]

print(arr[1:3]) # [1 3] - - 인덱스 1,2 선택
print(arr[1::2]) # [1 3] - - 1부터 2칸 간격

# 2차원 슬라이싱

arr = np.arange(25).reshape(5, 5)

# [[ 0  1  2  3  4]
#  [ 5  6  7  8  9]
# [10 11 12 13 14]
# [15 16 17 18 19]
# [20 21 22 23 24]]

print(arr[1, 1:5]) # [6 7 8 9] - - 2행의 2~5열
print(arr[:, 0:2]) # 첫번째와 두번째 컬럼 전체

# [[ 0  1]
#  [ 5  6]
# [10 11]
# [15 16]
```


[20 21]]

슬라이싱 문법	의미
arr[a:b]	인덱스 a부터 b - 1까지 선택
arr[a::n]	인덱스 a부터 n칸 간격으로 선택
arr[:, a:b]	모든 행에서 a열부터 b - 1열까지
arr[a:b, c:d]	a~b - 1행, c~d - 1열 선택 (부분 행렬)

11. 조건 필터링 (Boolean Indexing)

[개념] 불리언 인덱싱은 조건식을 배열에 적용하여 조건을 만족하는 원소만 추출하는 방법입니다. 여러 조건은 & (AND), | (OR) 연산자로 결합합니다.

조건 필터링 예제

```
import numpy as np

arr = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]

# 단일 조건

print(arr[arr > 5]) # [6 7 8 9]

print(arr[arr % 2 == 0]) # [0 2 4 6 8] 짝수만

# 여러 조건 (& = AND, | = OR)

print(arr[(arr > 3) & (arr < 7)]) # [4 5 6]

print(arr[(arr < 3) | (arr > 7)]) # [0 1 2 8 9]
```

[주의] 주의: 여러 조건을 결합할 때 반드시 각 조건을 괄호로 감싸야 합니다. 예) arr[(arr > 3) & (arr < 7)]
- - 괄호 필수!

12. 주요 개념 정리

개념	설명
ndarray	넘파이 다차원 배열 클래스. 벡터화 연산과 효율적 데이터 조작에 최적화

개념	설명
벡터화 연산	반복문 없이 배열 전체에 일괄 연산. 엑셀 수식 드래그와 유사
reshape	원소 수 유지하며 배열 형태 변경. 원소 수 불일치 시 ValueError
dtype	배열 원소의 데이터 타입 (int64, float64, str 등)
astype	배열의 데이터 타입을 변환하는 메서드
인덱싱	arr[행, 열] 형태로 특정 위치 접근. 열 단위 인덱싱 가능
슬라이싱	arr[a:b] 형태로 연속 구간 선택. 끝 인덱스 미포함
조건 필터링	arr[arr > 5] 형태로 조건 만족 원소 추출. & 로 다중 조건

[실습] 실습 팁: 주피터 노트북(Jupyter Notebook)에서 각 코드 예제를 직접 실행해 보세요.
아나콘다(Anaconda)를 사용하면 넘파이가 이미 설치되어 있어 별도 설치 없이 바로 사용 가능합니다.

넘파이(NumPy) 연습문제

총 20문제 | wikidocs 125545 기반

Part A. 배열 생성

Q1. 넘파이 импорт

넘파이 모듈을 np라는 이름으로 импорт하세요.

힌트: import ... as ...

내 답안:

Q2. ndarray 변환

다음 리스트를 넘파이의 ndarray 객체로 변환하세요.

```
data = [1, 2, 3]
```

힌트: np.array()

내 답안:

Q3. arange — 정수 범위

0부터 99까지의 정수가 저장된 ndarray 객체를 만드세요.

힌트: np.arange()

내 답안:

Q4. arange — 구구단

구구단 3단의 결과가 저장된 ndarray 객체를 만드세요.

```
# 기대 결과: [ 3 6 9 12 15 18 21 24 27]
```

힌트: np.arange(시작, 끝, 간격)

내 답안:

Q5. arange — 짝수

1000보다 작은 짝수가 저장된 ndarray 객체를 만드세요.

힌트: np.arange(시작, 끝, 간격)

내 답안:

Q6. arange — 실수 간격

0부터 1까지 0.1 간격으로 구성된 ndarray를 생성하세요. (1은 포함되지 않습니다)

힌트: np.arange(0, 1, 0.1)

내 답안:

Q7. ones — 1차원

1이 20개 저장된 아래의 ndarray 객체를 만드세요.

```
# 기대 결과: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

힌트: np.ones()

내 답안:

Q8. ones — 2차원

4행 5열로 저장된 아래의 ndarray 객체를 만드세요.

```
# 기대 결과:

# [[1 1 1 1 1]
# [1 1 1 1 1]
# [1 1 1 1 1]
# [1 1 1 1 1]]
```

힌트: np.ones((행, 열), dtype=int)

내 답안:

Q9. linspace

0~10 사이를 20개의 포인트로 균등 분할한 결과를 ndarray로 표현하세요.

힌트: `np.linspace(시작, 끝, 개수)`

내 답안:

Part B. reshape

Q10. reshape — 1차원 - > 2차원

다음 리스트를 2행 3열의 ndarray로 변환하세요.

```
data = [0, 1, 2, 3, 4, 5]
```

```
# 기대 결과:
```

```
# [[0 1 2]
```

```
# [3 4 5]]
```

힌트: `np.array(data).reshape(행, 열)`

내 답안:

Q11. reshape — 2차원 - > 1차원

다음 2차원 ndarray를 1차원 ndarray로 변환하세요.

```
arr = np.array([[0, 1, 2], [3, 4, 5]])
```

힌트: `.reshape(6)` 또는 `.reshape(-1)`

내 답안:

Q12. reshape — 열벡터

다음 ndarray를 아래와 같이 변경하세요.

```
arr = np.arange(3)
```

```
# 기대 결과:
```

```
# [[0]
```

```
# [1]
```

```
# [2]]
```

힌트: `.reshape(3, 1)`

내 답안:

Q13. 에러의 원인

다음 코드에서 에러가 발생하는 원인을 설명하세요.

```
np.arange(6).reshape(2, 4)  
  
# ValueError: cannot reshape array of size 6 into shape (2,4)
```

힌트: 원소 수: 변환 전 vs 변환 후

내 답안:

Part C. dtype / astype

Q14. dtype 확인 — 정수

다음 배열의 데이터 타입을 확인하세요.

```
arr = np.array([1, 2, 3])
```

힌트: arr.dtype

내 답안:

Q15. dtype 확인 — 문자열

다음 배열의 데이터 타입을 확인하세요.

```
arr = np.array(['1', '2', '3'])
```

힌트: arr.dtype

내 답안:

Q16. dtype — 형변환

다음 데이터를 np.int16 타입의 배열로 형변환하세요.

```
arr = np.array([1, 2, 3])
```

힌트: np.array(arr, dtype=np.int16)

내 답안:

Q17. astype — 타입 변경

arr 배열은 float64 타입입니다. 이를 np.int32 타입으로 변경하세요.

```
arr = np.array([1.1, 2.1, 3.1])
```

힌트: `.astype(np.int32)`

내 답안:

Part D. 인덱싱 / 슬라이싱 / 조건 필터링

Q18. 인덱싱 — 행/열

다음 ndarray에서 마지막 행([6,7,8])과 두 번째 열([1,4,7])을 각각 출력하세요.

```
arr = np.arange(9).reshape(3, 3)

# [[0 1 2]
#  [3 4 5]
#  [6 7 8]]
```

힌트: arr[행인덱스] / arr[:, 열인덱스]

내 답안:

Q19. 슬라이싱

아래 배열을 이용해 (1) [1 3] 슬라이싱, (2) [6 7 8 9] 슬라이싱, (3) 첫째 · 둘째 컬럼 슬라이싱을 하세요.

```
arr = np.arange(25).reshape(5, 5)

# [[ 0  1  2  3  4]
#  [ 5  6  7  8  9]
#  [10 11 12 13 14]
#  [15 16 17 18 19]
#  [20 21 22 23 24]]
```

힌트: arr[행, 시작:끝] / arr[:, 0:2]

내 답안:

Q20. 여러 조건 필터링

다음 배열에서 3보다 크고 7보다 작은 수만 추출하세요.

```
arr = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]
```

힌트: arr[(arr > 조건1) & (arr < 조건2)]

내 답안:

정답 모음

각 문제의 정답 코드입니다.

Q1. 넘파이 임포트	<code>import numpy as np</code>
Q2. ndarray 변환	<code>arr = np.array(data) # arr.dtype: int64</code>
Q3. arange 정수	<code>arr = np.arange(100)</code>
Q4. arange 구구단	<code>arr = np.arange(3, 30, 3)</code>
Q5. arange 짝수	<code>arr = np.arange(0, 1000, 2)</code>
Q6. arange 실수	<code>arr = np.arange(0, 1, 0.1)</code>
Q7. ones 1차원	<code>arr = np.ones(20, dtype=int)</code>
Q8. ones 2차원	<code>arr = np.ones((4, 5), dtype=int)</code>
Q9. linspace	<code>arr = np.linspace(0, 10, 20)</code>
Q10. reshape 2차원	<code>arr = np.array(data).reshape(2, 3)</code>
Q11. reshape 1차원	<code>arr2d.reshape(6) # 또는 reshape(-1)</code>
Q12. reshape 열벡터	<code>arr.reshape(3, 1)</code>
Q13. 에러 원인	<code>arange(6) -> 원소 6개, reshape(2,4) -> 8개 필요 / 원소 수 불일치 -> ValueError 발생</code>
Q14. dtype 정수	<code>print(arr.dtype) # int64</code>
Q15. dtype 문자열	<code>print(arr.dtype) # <U1</code>
Q16. dtype 형변환	<code>arr_int16 = np.array(arr, dtype=np.int16)</code>
Q17. astype	<code>print(arr.astype(np.int32)) # [1 2 3]</code>
Q18. 행/열 인덱싱	<code>print(arr[2]) # 마지막 행: [6 7 8] / print(arr[:, 1]) # 두 번째 열: [1 4 7]</code>
Q19. 슬라이싱	<code>arr_1d = np.arange(5); print(arr_1d[1::2]) # [1 3] / print(arr[1, 1:5]) # [6 7 8 9] / print(arr[:, 0:2]) # 첫째,둘째 컬럼</code>
Q20. 조건 필터링	<code>print(arr[(arr > 3) & (arr < 7)]) # [4 5 6]</code>

[완료] 수고하셨습니다! 넘파이의 기초 개념부터 조건 필터링까지 모두 학습했습니다. 판다스(Pandas) 학습 시 넘파이 개념이 기반이 되니 꼭 복습해 두세요.